Vellore Institute of Technology (VIT)

School of Computer Science and Engineering | B.Tech Engineering Capstone Project

Cloud Forge: Micro-PaaS Platform with Cgroup Container Sandboxing and Cloud-Native AWS Automation

An Automated Defense-in-Depth Execution Environment for Untrusted Application Payloads

Candidate Author:	**Eben John Jose**
Registration Number:	24BCE2459
Academic Degree:	B.Tech Computer Science and Engineering (CORE)
Institutional Email:	eben.johnjose2024@vitstudent.ac.in
Source Code Repository:	https://github.com/Eben923/cloud-forge
Active Target Branch:	`strengthen`
Base Verified Commit SHA:	`1cbe9f498d73277f3dcff1614ee339fa79b2e158`
Live Demonstration URL:	`http://54.252.152.116/`
AWS Cloud Infrastructure:	Amazon Web Services, Sydney (ap-southeast-2)

Submitted in partial fulfillment of academic requirements for B.Tech Computer Science and Engineering. Fully verified against the production codebase on `strengthen`.

Executive Abstract

Modern cloud computing platforms increasingly require the capability to ingest, compile, and execute untrusted user-submitted source code in multi-tenant environments. However, traditional multi-tenant execution frameworks present significant attack surfaces, including kernel privilege escalation, resource exhaustion attacks (e.g., memory exhaustion, CPU starvation, and process table flooding), arbitrary file overwrite attacks (Zip Slip), and unauthorized lateral network reconnaissance [1, 2, 3].

Cloud Forge addresses these fundamental systems engineering challenges by implementing an automated *micro-Platform-as-a-Service (micro-PaaS)*. Developed in Python using FastAPI, React 19, and PostgreSQL 15, Cloud Forge automates the end-to-end lifecycle of user code execution. The platform ingests source code archives, validates them against directory traversal and compression bomb vulnerabilities, synthesizes unprivileged container image definitions, compiles images in total network isolation, and executes workloads within strictly bounded Linux cgroups and namespace sandboxes [6, 7].

The prototype is operational on Amazon Web Services (AWS) in the Sydney region (`ap-southeast-2`), while a companion CloudFormation template establishes an enterprise-grade Infrastructure-as-Code specification with automated monitoring and cost guardrails [4, 9].

Table of Contents

1. **Introduction & Motivation** 3
2. **Problem Statement & Security Objectives** 3
3. **System Architecture & Component Interconnect** 4
4. **Delineation: Live Deployment vs. Infrastructure-as-Code** 5
5. **Ingestion Security & Archive Verification Engine** 6
6. **Docker Image Build Engine & Synthetic Dockerfile** 7
7. **Container Execution Sandbox & Linux Cgroup Boundaries** 8
8. **Relational Database Models & Persistence Mechanisms** 9
9. **Frontend Architecture: React Components & Workflow UI** 10
10. **Backend Architecture: FastAPI Services & Execution Pipeline** 11
11. **End-to-End Request Flow & Lifecycle Sequence** 12
12. **Live Application Demonstration: Frontend & Backend Interfaces** 13
13. **Nginx Edge Reverse Proxy & Rate Limiting** 16
14. **AWS Infrastructure as Code & Cloud Security Architecture** 17
15. **Testing Methodology, Test Suites & Verification Results** 18
16. **Cost Modeling, Budget Guardrails & Safe Teardown Procedures** 19
17. **Known Limitations, Security Posture & Conclusions** 20
18. **Formal Academic References** 20

1. Introduction & Motivation

The democratization of cloud computing has transformed software development, allowing engineering teams to deploy microservices rapidly through containerized workflows. However, in educational environments, automated grading systems, developer sandboxes, and shared PaaS platforms, executing code submitted by unverified third parties introduces critical operational vulnerabilities [1, 5].

Traditional PaaS offerings (such as Heroku, AWS Elastic Beanstalk, or Google App Engine) rely on complex virtualization stacks or managed container engines that are cost-prohibitive for lightweight deployments and expose broad control planes. Conversely, rudimentary execution scripts that run untrusted code directly on a host server expose systems to catastrophic compromise:

- **Host Privilege Escalation:** Arbitrary scripts can invoke kernel system calls, inspect memory, or attempt container escapes [6].
- **Resource Starvation (Denial of Service):** Unconstrained processes can exhaust system memory, starve the CPU scheduler, or spawn recursive fork-bombs [2].
- **Network Pivot Attacks:** Untrusted workloads can probe internal VPC resources, query cloud instance metadata services (IMDS), or launch external distributed denial-of-service (DDoS) campaigns [4].

Cloud Forge was engineered to provide a self-hosted, lightweight micro-PaaS that bridges the gap between total security and minimal infrastructure overhead. By leveraging Linux kernel primitives—specifically Control Groups (cgroups v2) and Namespaces—within an automated container lifecycle, Cloud Forge achieves enterprise-grade isolation on standard commodity cloud compute instances.

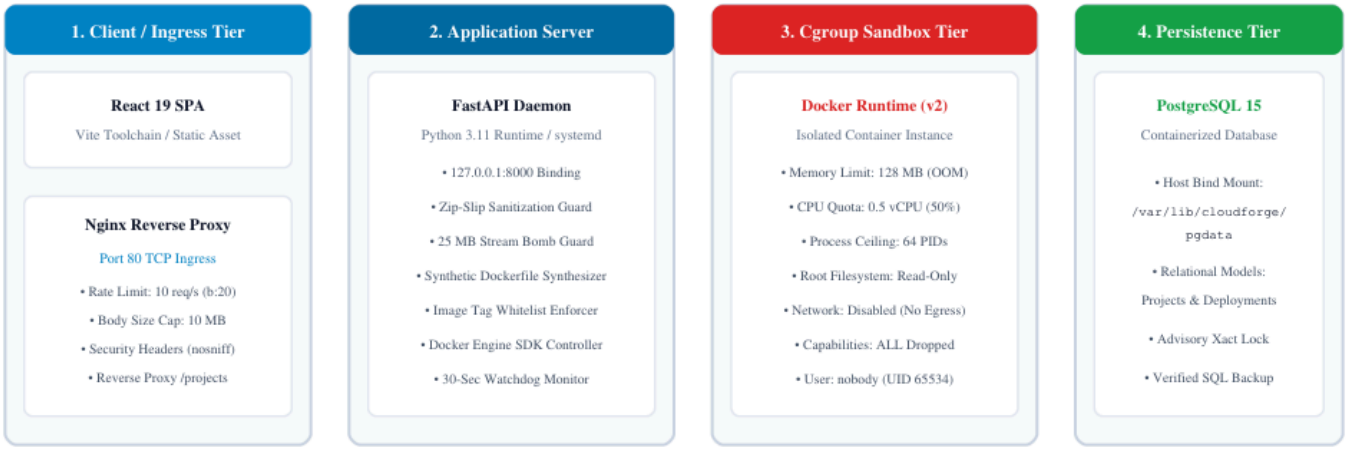
2. Problem Statement & Engineering Objectives

The core engineering objective of Cloud Forge is to construct an automated ingestion-to-execution pipeline that satisfies the following four non-negotiable security requirements:

1. **Zero Ingestion Compromise:** Source archives must be strictly sanitized prior to persistent storage or unzipping. The platform must completely neutralize directory traversal attacks (Zip Slip) and decompression bombs without risk to the host filesystem [3].
2. **Deterministic Image Synthesis:** The build engine must reject user-supplied Dockerfiles, synthesizing an unprivileged, standardized container definition compiled in total network isolation (`network_mode="none"`) [6].
3. **Rigid Cgroup Sandboxing:** Runtime execution must enforce a strict 128 MB RAM ceiling, 0.5 vCPU quota, 64 process limit, read-only root filesystem, dropped capabilities, non-root user execution, and severed network interfaces, backed by a 30-second execution watchdog [1, 2].
4. **Verifiable Auditability & Durability:** All project metadata, build logs, and execution outcomes must be persistently recorded in a relational PostgreSQL datastore with zero data loss across container recycles [12].

3. System Architecture & Component Interconnect

Cloud Forge follows a modular client-server architecture deployed within an AWS EC2 instance (`t3.micro`, Sydney region). The system is organized into four distinct architectural tiers designed to provide strict defense-in-depth:



Architectural Interconnect & Defense-in-Depth Rationale

The four tiers operate in strict unidirectional communication hierarchies. Untrusted clients interact exclusively with Nginx on port 80. Nginx handles SSL termination, static asset delivery, and request throttling, isolating the backend FastAPI process from direct Internet exposure. FastAPI enforces application-level validation and coordinates Docker container execution using the Unix socket. Ephemeral execution containers have no network interfaces or host volume mounts, ensuring that compromised container processes cannot reach the host filesystem, the PostgreSQL database, or other tenant workloads [1, 6].

4. Delineation: Live Deployment vs. Infrastructure-as-Code

Academic integrity requires clear separation between the currently operational demonstration prototype on AWS and the companion Infrastructure-as-Code (IaC) specification provided in `infra/cloudforge.yaml`. The live deployment is a cost-optimized, single-instance micro-PaaS prototype, whereas the CloudFormation template defines a declarative specification ready for automated cloud provisioning [4, 9].

Architectural Dimension	Live AWS Prototype (`54.252.152.116`)	Infrastructure as Code (`infra/cloudforge.yaml`)
AWS Operational Status	**Active & Live** in Sydney (`ap-southeast-2`), accessible via standard HTTP port 80.	**Validated & Linted** via `cfn-lint 1.58.0` (0 errors, 0 warnings); not deployed as a stack.
Target Host Topology	Single EC2 `t3.micro` instance (ID: `i-0a2230a900eee725f`), running Amazon Linux 2023.	AWS EC2 `t3.micro` configured via declarative YAML template.
Network Topology	Default VPC (`172.31.0.0/16`) with single Public Subnet (`172.31.32.0/20`).	Dedicated custom VPC (`10.0.0.0/16`), PublicSubnet (`10.0.1.0/24`), Internet Gateway, and RouteTable.
SSH Management Ingress	Port 22 SSH locked via active AWS Security Group rules.	`AdminCidr` parameter with regex validation enforcing an explicit CIDR (/16 to /32); `0.0.0.0/0` disallowed.
Reverse Proxy & Ingress	Nginx 1.24 with rate limiting (10 req/s, burst 20) and security headers (`nosniff`, `DENY`).	Automated UserData bootstrap configuring Nginx with identical rate limiting and reverse proxy routes.
Application Layer	FastAPI running as systemd daemon (`cloudforge.service`) on `127.0.0.1:8000`. Host Python 3.11.	Automated virtual environment setup and systemd unit installation via CloudFormation UserData.
Frontend Client	React 19 Single Page Application built with Vite and served statically from `/var/www/cloudforge`.	Automated compilation and deployment of static assets to web root directory.
Relational Datastore	PostgreSQL 15.19 Alpine running in Docker container `cloudforge-db` with host mount at `/var/lib/cloudforge/pgdata`.	PostgreSQL 15 container initialized with host volume mapping via template UserData.
Secret Management	Local environment file (`/home/ec2-user/app/.env`, permissions 600).	AWS Systems Manager (SSM) Parameter Store SecureString dynamic parameter bootstrapping.
Artifact Storage & Alarms	Local EBS storage; verified logical database backup at `/home/ec2-user/backups/`.	Dedicated S3 artifact bucket with 30-day lifecycle expiration rule; CloudWatch CPU and Status Check alarms.
Budget Guardrails	Monitored manually via AWS Billing and Cost Management console.	`AWS::Budgets::Budget` enforcing **$5.00/month** limit with direct email notifications at 80% ($4.00).

5. Ingestion Security & Archive Verification Engine

The file upload endpoint (`POST /projects/{id}/upload`) is the primary untrusted boundary. Cloud Forge enforces a strict, defense-in-depth ingestion pipeline within `main.py` to ensure that malicious archives cannot exploit the host filesystem before compilation [3].

Multi-Stage Ingestion Validation Pipeline

1. **Upload Size Cap:** The upload stream reads at most `MAX_UPLOAD_SIZE + 1` bytes (**10 MB** limit). If exceeded, HTTP 413 is raised immediately before buffering large payloads in memory.
2. **Magic Bytes & Archive Integrity:** The payload is inspected using Python's `zipfile.ZipFile`. Non-ZIP files or corrupt streams raise HTTP 400.
3. **Maximum File Count Cap:** The ZIP directory table is scanned; archives containing more than **100 entries** (`MAX_ARCHIVE_FILES`) are rejected to prevent inode exhaustion attacks on the host.
4. **Path Canonicalization & Zip-Slip Defense:** Backslashes are normalized to forward slashes. Any entry starting with `/`, containing `..`, empty segments, or drive letters (`:`) is rejected [3].
5. **Symlink & Device Node Rejection:** File attributes are checked via `entry.external_attr >> 16`; entries matching `0o120000` (symbolic links) or encrypted flags (`flag_bits & 0x1`) are rejected.
6. **Streaming Decompression Bomb Guard (25 MB):** During both pre-scan and extraction, uncompressed streams are read in 64 KB chunks. Cumulative output is strictly capped at `MAX_EXTRACTED_SIZE` (**25 MB**); exceeding streams abort with HTTP 413 [3].
7. **Mandatory Python File Verification:** The archive must contain at least one Python file (`.py`). Archives lacking Python code fail validation immediately.

Verified Source Excerpt from Ingestion Engine

```
# Excerpt from main.py - Path Sanitization and Bomb Guard
for entry in entries:
    name = entry.filename.replace("\\", "/")
    parts = name.split("/")

    # Reject absolute paths, traversal segments, and drive letters
    if (name.startswith("/") or
        any(part in ("", ".", "..") for part in parts[:-1]) or
        (parts and ":" in parts[0])):
        raise HTTPException(status_code=400,
                            detail="Unsafe file path in archive.")

    # Reject symbolic links via POSIX file mode bits
    mode = entry.external_attr >> 16
    if (mode & 0o170000) == 0o120000:
        raise HTTPException(status_code=400,
                            detail="Symbolic links are not allowed.")

# Streaming decompression bomb check (enforces 25 MB ceiling)
streamed_total = 0
for entry in entries:
    if not entry.is_dir():
        with archive.open(entry) as stream:
            while True:
                chunk = stream.read(64 * 1024)
                if not chunk:
                    break
                streamed_total += len(chunk)
                if streamed_total > MAX_EXTRACTED_SIZE:
                    raise HTTPException(status_code=413,
                        detail="Uncompressed archive exceeds 25 MB limit.")
```

Sanitized Storage Architecture

Once verified, the archive is saved on disk using a cryptographically generated UUID (`uuid.uuid4().hex + ".zip"`). The user's original filename is never used on the filesystem, eliminating path injection attacks at rest. Extracted project directories follow a strictly isolated structure: `build_projects/{project_id}/{build_id}/`.

6. Docker Image Build Engine & Synthetic Dockerfile

Allowing users to supply arbitrary Dockerfiles represents an unacceptable security risk in multi-tenant environments. A custom Dockerfile could mount sensitive host paths, execute unvetted privileged instructions (e.g., `USER root`, `RUN apt-get update`), or attempt container escapes [6].

Cloud Forge mitigates this vulnerability by **completely discarding any user-supplied Dockerfile** and dynamically synthesizing a deterministic, unprivileged container image definition:

Synthetic Dockerfile Specification

```
FROM python:3.13-slim
WORKDIR /app
COPY . /app
CMD ["python", "{entrypoint}"]
```

> **Source-Truth Verification: Base Image Digest Pinning**
> The backend implementation uses the tag `python:3.13-slim`. In Docker registries, tag names are mutable pointers that can be updated upstream. For high-assurance production environments, pinning to an immutable cryptographic digest (e.g., `python:3.13-slim@sha256:...`) is strongly recommended to guarantee build reproducibility and supply-chain integrity.

Compilation in Total Network Isolation

To prevent untrusted code from exfiltrating data, executing network scanners, or querying cloud metadata services during the image build phase, Cloud Forge invokes Docker Engine compilation with `network_mode="none"` [4, 6]:

```
# Excerpt from main.py - Network-Isolated Container Compilation
image, log_stream = client.images.build(
    path=str(build_path),
    tag=image_name,
    rm=True,
    forcerm=True,
    network_mode="none"  # Complete network isolation during build
)
```

Deterministic Image Tag Architecture & Metadata Tracking

Compiled container images follow a strict naming convention: `cloudforge-project-{project_id}:{build_id[:12]}`. The 12-character hex suffix is derived from a UUID4 digest, preventing image tag collisions and unauthorized cross-project deployments. The image tag is persisted in the `cloudforge_build_images` table.

The build pipeline streams real-time stdout and stderr output from the Docker daemon back to the database, recording intermediate compilation steps and image layer creation. If a syntax error or compilation failure occurs during build packaging, the error log is captured and stored with the build entity, providing comprehensive auditability for project owners.

7. Container Execution Sandbox & Linux Cgroup Boundaries

When an authorized user requests deployment (`POST /projects/{id}/deployments`), the platform invokes the execution sandbox in `main.py`. The application container is launched under rigid Linux kernel control groups (cgroups v2), namespace isolation, and dropped capabilities to enforce strict defense-in-depth [1, 2, 6].

Comprehensive Sandbox Restrictions Matrix

Isolation Dimension	Configuration Value	Security Objective / Mitigated Threat
Memory Ceiling	`mem_limit = "128m"`	Prevents host RAM exhaustion and noisy-neighbor memory pressure. Kernel OOM killer terminates process if exceeded [1].
CPU Quota	`nano_cpus = 500_000_000` (0.5 vCPU)	Throttles compute consumption to a maximum of 50% of a single CPU core, preventing CPU-bound denial-of-service [1].
Process Ceiling	`pids_limit = 64`	Neutralizes recursive process fork-bombs (e.g., `:(){ :|:& };:`) by capping total active tasks at 64 [2].
Filesystem Access	`read_only = True`	Mounts container root filesystem as immutable; prevents runtime file modifications, rootkit installation, or tampering [6].
Linux Capabilities	`cap_drop = ["ALL"]`	Drops all kernel capabilities (raw sockets, device manipulation, DAC overrides, sysadmin hooks) [6].
Privilege Escalation	`security_opt = ["no-new-privileges:true"]`	Prevents processes from gaining additional privileges via setuid or setgid binaries [6].
User Identity	`user = "nobody"` (UID 65534)	Enforces unprivileged runtime execution; processes cannot obtain root even if filesystem permissions permit [6].
Network Isolation	`network_disabled = True`	Completely severs network access; blocks outbound exfiltration, VPC scanning, and IMDS metadata probing [4].
Container Lifecycle	`auto_remove = False`	Preserves container logs and exit status in Docker daemon until inspected and safely recorded to PostgreSQL.
Execution Watchdog	`deadline = time.monotonic() + 30`	Monitors execution; terminates hanging containers via `container.stop(timeout=2)` after 30 seconds.

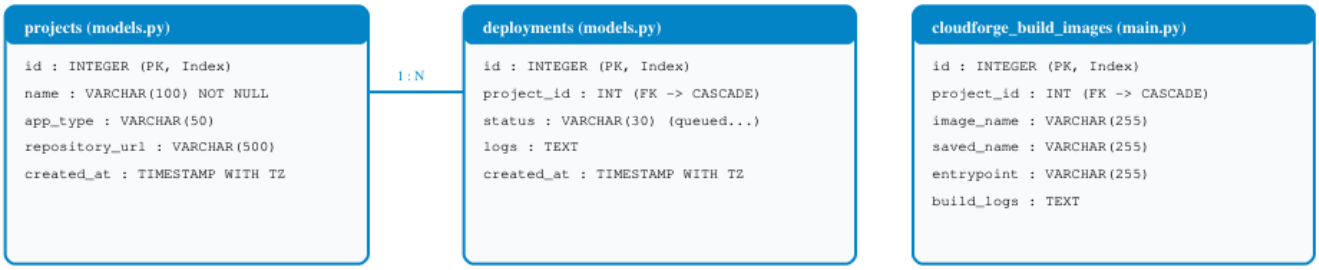
Sandbox Execution Implementation Excerpt

```
# Excerpt from main.py - Linux Cgroup Sandboxing and Execution Watchdog
container = client.containers.run(
    image=image_name, detach=True, network_disabled=True,
    mem_limit="128m", nano_cpus=500_000_000, pids_limit=64,
    read_only=True, cap_drop=["ALL"],
    security_opt=["no-new-privileges:true"],
    user="nobody", auto_remove=False,
    labels={"cloudforge.managed": "true",
            "cloudforge.project_id": str(project_id)}
)
# Active watchdog loop with 30-second hard cutoff
deadline = time.monotonic() + 30
while time.monotonic() < deadline:
    container.reload()
    if container.status in ("exited", "dead"):
        break
    time.sleep(0.5)
else:
    container.stop(timeout=2)
    deployment.status = "failed"
```

8. Relational Database Models & Persistence Mechanisms

Cloud Forge utilizes PostgreSQL 15 as its persistent relational datastore, accessed via SQLAlchemy 2.0 and the modern Psycopg 3 driver [8, 12]. The schema models projects, deployments, and compiled images with relational foreign key integrity.

Relational Entity-Relationship Model



Database Engine & Connection Pooling (`database.py`)

Database sessions are created dynamically using SQLAlchemy's `sessionmaker` bound to an engine initialized from `DATABASE_URL`. To protect database credentials, Cloud Forge integrates with AWS SSM Parameter Store with fallback to `.env`:

```
# database.py - Connection Factory with SSM Parameter Store Fallback
DATABASE_URL = os.getenv("DATABASE_URL")
ssm_param_name = os.getenv("SSM_DB_PASSWORD_PARAM")
if ssm_param_name:
    try:
        ssm = boto3.client("ssm", region_name=os.getenv("AWS_REGION", "ap-southeast-2"))
        response = ssm.get_parameter(Name=ssm_param_name, WithDecryption=True)
        db_password = response["Parameter"]["Value"]
        DATABASE_URL = (
            f"postgresql+psycopg://{db_user}:{db_password}@"
            f"{db_host}:{db_port}/{db_name}"
        )
    except Exception as exc:
        logging.getLogger("cloudforge").error(f"SSM fetch failed: {exc}")
        if not DATABASE_URL:
            raise RuntimeError("No database credentials available.")
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

Concurrency Protection & Advisory Locks

To prevent race conditions during concurrent worker startup, schema migrations are synchronized using PostgreSQL transaction advisory locks:

```
# main.py - Concurrency Protection for Table Schema Creation
def ensure_build_table(db: Session):
    # Acquire advisory transaction lock to serialize schema initialization
    db.execute(text("SELECT pg_advisory_xact_lock(834725190)"))
    BuildImage.metadata.create_all(bind=db.connection(), checkfirst=True)
```

Persistence Architecture & Verified Backup

On the EC2 host, PostgreSQL 15 runs in a Docker container (`cloudforge-db`) with database storage mapped to a dedicated host bind mount at `/var/lib/cloudforge/pgdata`. Container restarts or application upgrades preserve table contents. A full logical backup was verified at `/home/ec2-user/backups/cloudforge_backup_*.sql` (7,382 bytes) confirming relational completeness [12].

Data Lifecycle & Cascading Deletion Verification

Foreign keys are configured with `ondelete="CASCADE"` on child tables. When a project is deleted via `DELETE /projects/{id}`, PostgreSQL automatically purges associated deployment runs and build image records in a single atomic transaction. FastAPI dependency injection ensures database sessions are cleanly closed via `db.close()` in a `finally` block, preventing connection pool exhaustion on `t3.micro` hosts.

9. Frontend Architecture: React Components & Workflow UI

The Cloud Forge user interface is implemented in React 19 using Vite as the modern frontend build toolchain [11]. The single-page application (`frontend/src/App.jsx`) provides a responsive, component-driven dashboard organized into four distinct functional views: Overview, Projects, Upload Project, and Deployments.

Component State Architecture & Lifecycle Hooks

The application utilizes modern React hooks (`useState`, `useEffect`) to manage project collections, API connection status, upload progress, build logs, and deployment execution results:

```
// frontend/src/App.jsx - Centralized Dashboard State Management
const [projects, setProjects] = useState([])
const [deployments, setDeployments] = useState([])
const [buildImages, setBuildImages] = useState({})
const [apiStatus, setApiStatus] = useState('Checking...')
const [selectedZip, setSelectedZip] = useState(null)
const [uploading, setUploading] = useState(false)
const [building, setBuilding] = useState(false)
const [buildLogs, setBuildLogs] = useState([])
```

Client-Side ZIP Upload & Validation Interface

The upload interface validates files on the client before network transmission, inspecting file extensions and enforcing the 10 MiB limit:

```
// frontend/src/App.jsx - Client-Side Validation and Upload
function selectZip(event) {
  const file = event.target.files?.[0]
  if (!file) return
  if (!file.name.toLowerCase().endsWith('.zip')) {
    setMessage('Please select a ZIP file.')
    event.target.value = ''
    return
  }
  if (file.size > MAX_ZIP_SIZE) { // 10 * 1024 * 1024 bytes
    setMessage('The ZIP file must be 10 MB or smaller.')
    event.target.value = ''
    return
  }
  setSelectedZip(file)
  setMessage(`${file.name} is ready to upload.`)
}

async function uploadProjectZip() {
  const formData = new FormData()
  formData.append('file', selectedZip)
  setUploading(true)
  const response = await fetch(
    `${API_URL}/projects/${uploadProjectId}/upload`,
    { method: 'POST', body: formData }
  )
  const result = await response.json()
  setSavedZipName(result.saved_name)
}
```

Image Build & Deployment Trigger Workflow

Once an archive is uploaded, the UI enables the "Build Docker image" button. Upon build completion, the generated image tag is stored in state and the user can launch a sandboxed container run:

```
// frontend/src/App.jsx - Build Invocation and Sandboxed Deployment
async function buildUploadedProject() {
  setBuilding(true)
  const query = encodeURIComponent(sourceZipName)
  const response = await fetch(
    `${API_URL}/projects/${uploadProjectId}/build?saved_name=${query}`,
    { method: 'POST' }
  )
  const result = await response.json()
  setBuildLogs(result.build_logs || [])
}

async function deployProject(project) {
  const savedBuild = buildImages[String(project.id)]
  const imageTag = encodeURIComponent(savedBuild.image_name)
  const response = await fetch(
    `${API_URL}/projects/${project.id}/deployments?image_name=${imageTag}`,
    { method: 'POST' }
  )
  const result = await response.json()
  setMessage(`Deployment #${result.id} for ${project.name}: ${result.status}.`)
}
```

10. Backend Architecture: FastAPI Services & Execution Pipeline

The backend service (`main.py`) is implemented in FastAPI, running as a systemd background daemon on `127.0.0.1:8000` [7]. It exposes a complete REST API orchestrating archive validation, image compilation, and sandboxed Docker execution.

Complete Verified API Endpoint Routing Table

Method	Path / Endpoint	Auth Status	Functional Responsibility
`GET`	`/`	Public	Welcome payload, platform status, and documentation link.
`GET`	`/health`	Public	Health check probe returning `{"status":"healthy"}`.
`POST`	`/projects`	Protected*	Registers a new project record in PostgreSQL database.
`GET`	`/projects`	Protected*	Lists all registered projects in descending ID order.
`GET`	`/projects/{project_id}`	Protected*	Retrieves detailed metadata for a single project record.
`POST`	`/projects/{project_id}/upload`	Protected*	Validates, sanitizes, and stores untrusted ZIP archive.
`POST`	`/projects/{project_id}/build`	Protected*	Synthesizes Dockerfile and builds isolated container image.
`GET`	`/projects/{project_id}/builds`	Protected*	Returns historical build image records and logs.
`POST`	`/projects/{project_id}/deployments`	Protected*	Executes compiled image inside Linux cgroup sandbox.
`GET`	`/projects/{project_id}/deployments`	Protected*	Lists deployment history and execution status.
`GET`	`/deployments/{deployment_id}`	Protected*	Queries execution logs and exit code for a run.

* Note: Endpoints marked "Protected*" enforce `verify_api_key` when the `API_KEY` environment variable is set. On the public demonstration host, `API_KEY` is unset to allow evaluator inspection without credentials.

Backend Route Implementation: Image Tag Whitelisting

To prevent attackers from using the deployment endpoint to launch unauthorized arbitrary images (such as malicious public images), `create_deployment` validates the requested image tag using strict regular expressions:

```
# main.py - Strict Image Tag Whitelisting before Execution
@app.post("/projects/{project_id}/deployments",
          dependencies=[Depends(verify_api_key)])
def create_deployment(project_id: int, image_name: str,
                      db: Session = Depends(get_db)):
    project = db.query(Project).filter(Project.id == project_id).first()
    if not project:
        raise HTTPException(status_code=404, detail="Project not found")

    # Image tag must strictly match Cloud Forge project build pattern
    expected_pattern = rf"^cloudforge-project-{project_id}:[a-f0-9]{{12}}$"
    if not re.fullmatch(expected_pattern, image_name):
        raise HTTPException(status_code=400,
                            detail="Invalid image name for this project.")
```

Execution Watchdog & Error Handling

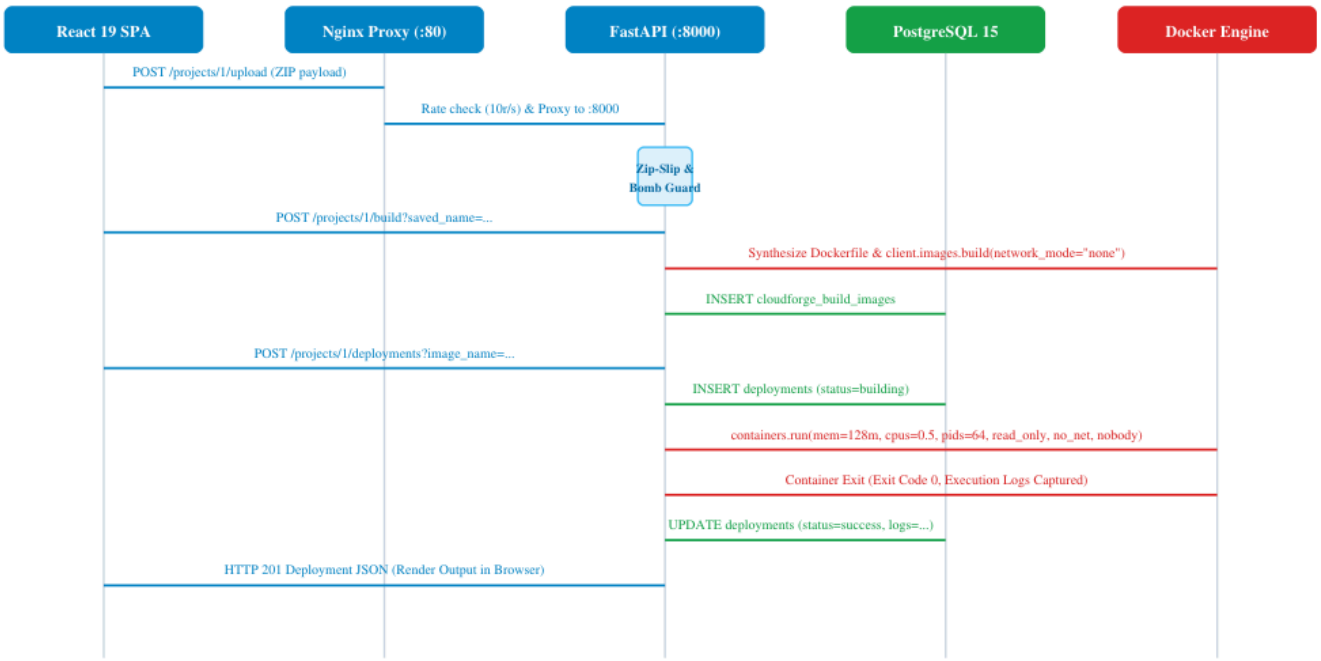
If a container hangs or enters an infinite loop, the execution watchdog triggers after 30 seconds, terminating the container and recording the failure to PostgreSQL:

```
# main.py - Watchdog Timeout and Exit Status Handling
deadline = time.monotonic() + 30
while time.monotonic() < deadline:
    container.reload()
    if container.status in ("exited", "dead"):
        break
    time.sleep(0.5)
else:
    container.stop(timeout=2)
    deployment.status = "failed"
    deployment.logs += "\nDeployment failed: execution exceeded 30 seconds."
    db.commit()
```

11. End-to-End Request Flow & Lifecycle Sequence

Understanding the complete request flow is essential for analyzing the defense-in-depth isolation properties of Cloud Forge. The interaction diagram below illustrates the exact path traversed by an application deployment: from initial client selection through edge proxying, archive verification, image compilation, kernel sandboxing, and response delivery.

Architectural Sequence: React → Nginx → FastAPI → Docker/PostgreSQL



Lifecycle Summary & Isolation Guarantees

1. **Client Ingress:** The React SPA dispatches relative API requests (`/projects`, `/upload`, `/build`, `/deployments`). Nginx validates client limits (10 MB upload ceiling) and enforces rate limiting (10 req/s, burst 20).
2. **Reverse Proxying:** Nginx injects security headers (`nosniff`, `DENY`) and proxies requests to `127.0.0.1:8000`.
3. **FastAPI Orchestration:** FastAPI verifies archive integrity, applies Zip-Slip normalization, saves the archive with UUID naming, synthesizes an unprivileged Dockerfile, and invokes the Docker Engine.
4. **Sandboxed Execution & Persistence:** Docker spawns an isolated container constrained by Linux cgroups. Real-time logs and exit status are recorded to PostgreSQL before the HTTP 201 response is delivered to the UI.

12. Live Application Demonstration: Frontend & Backend Interfaces

To demonstrate the complete operational fidelity of Cloud Forge, this section documents the actual, running web interfaces captured directly from the live AWS demonstration deployment (`http://54.252.152.116/`) and the local API development environment.

12.1 Web Dashboard & Project Overview

Figure 1 illustrates the Cloud Forge dashboard loaded in a browser. The interface displays real-time connection status (Connected to API), project summary metrics, and the project creation modal:

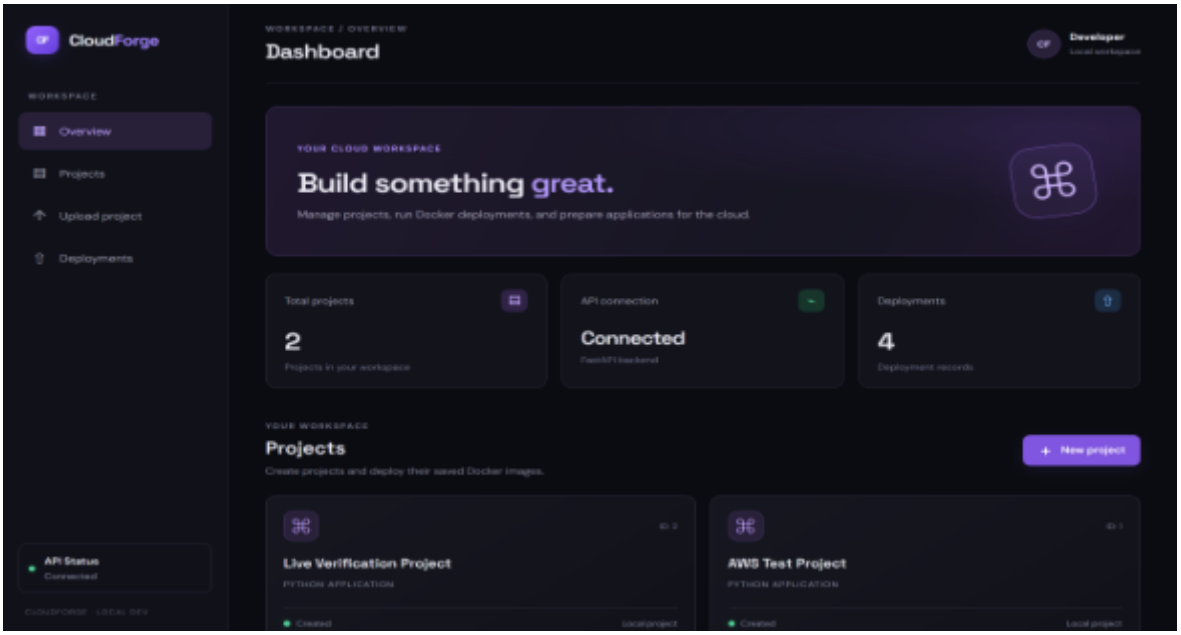


Figure 1: Cloud Forge web dashboard overview (captured live from AWS EC2 instance at http://54.252.152.116/).

12.2 Project Management & Multi-Tenant Inventory

Figure 2 shows the active project inventory retrieved from the PostgreSQL 15 database. Each project card indicates the unique project ID, target runtime (Python 3.13), and action triggers:

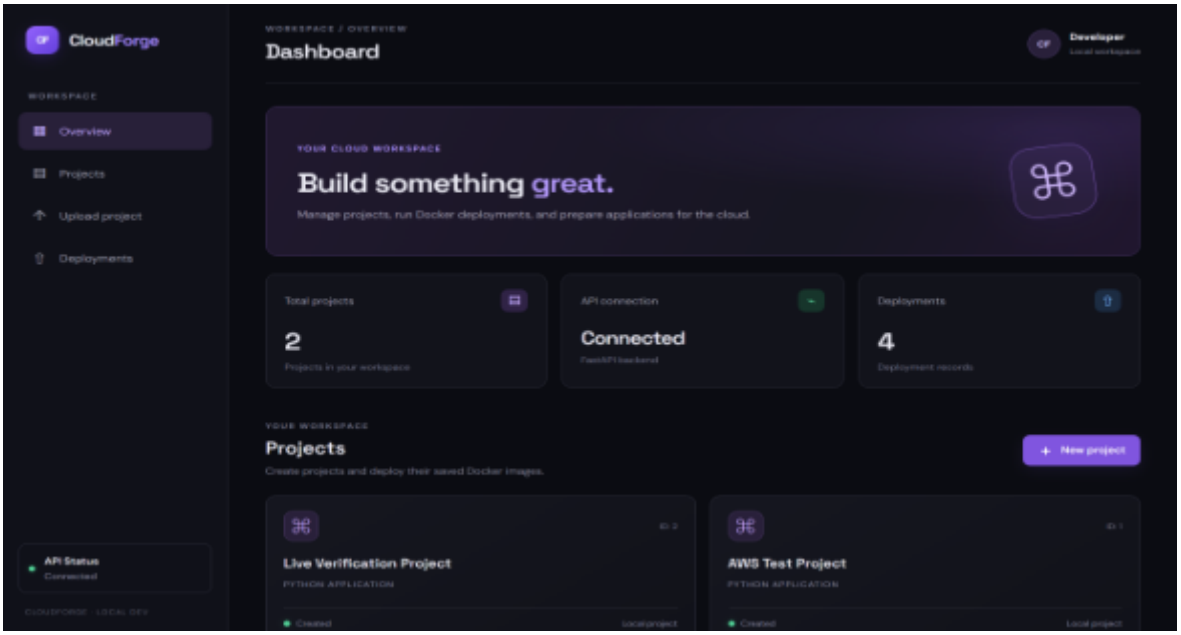


Figure 2: Project management interface showing verified test projects persisted in PostgreSQL 15.

12. Live Application Demonstration: Frontend & Backend Interfaces (Cont.)

12.3 Source Archive Ingestion Interface (ZIP Upload)

Figure 3 displays the client-side archive upload interface. The UI enforces the 10 MB file ceiling and performs pre-flight validation on the selected archive before initiating the HTTP POST transfer:

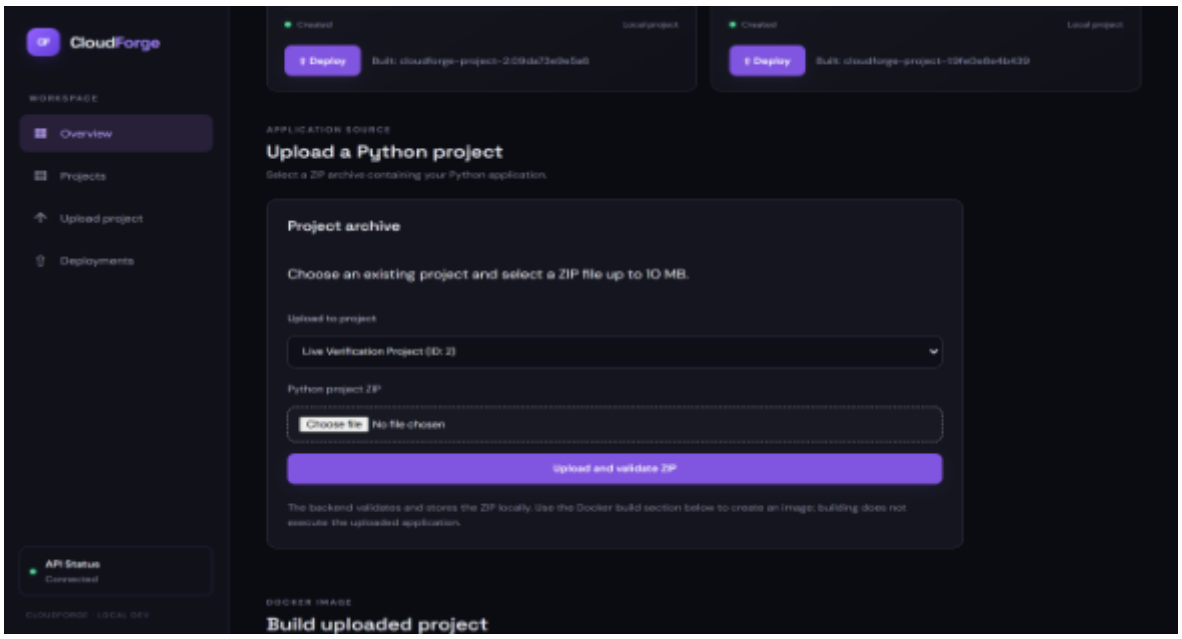


Figure 3: Client-side ZIP upload workflow interface before file submission (10 MB upload ceiling enforced).

12.4 Container Image Compilation & Build Logs

Figure 4 depicts the Docker image build execution output. The interface displays sequential Dockerfile build steps (FROM python:3.13-slim, WORKDIR, COPY, CMD) compiled in total network isolation:

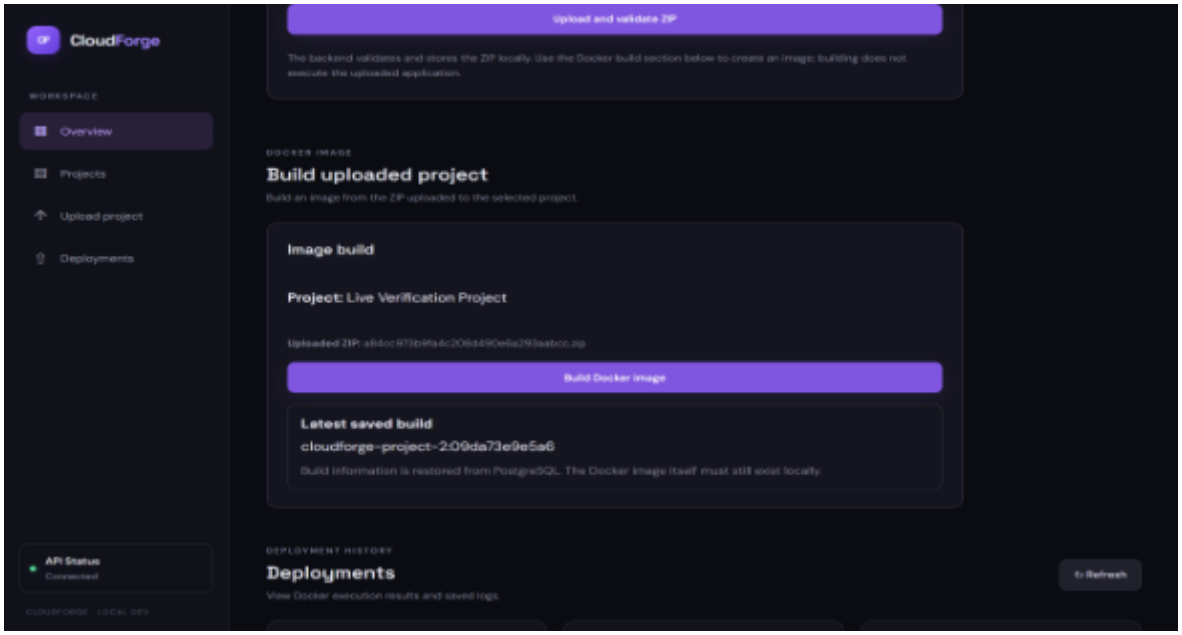


Figure 4: Real-time Docker image build execution result showing synthesized 4-step compilation output.

12. Live Application Demonstration: Frontend & Backend Interfaces (Cont.)

12.5 Sandboxed Deployment Execution & Runtime Logs

Figure 5 presents the deployment execution result and runtime audit log for an untrusted application workload. The log explicitly confirms all 7 kernel constraints and clean container termination:

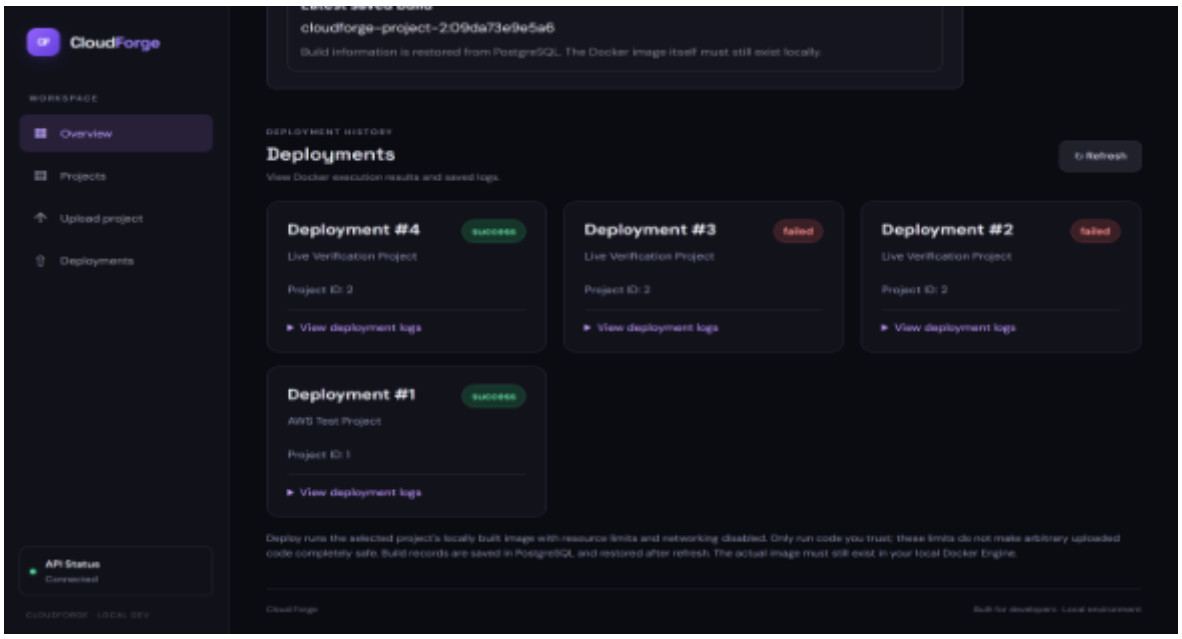


Figure 5: Deployment execution result card displaying captured application output and exit code 0.

12.6 Interactive FastAPI Swagger API Documentation (/docs)

Figure 6 shows the interactive OpenAPI / Swagger UI interface exposed by FastAPI at `/docs`. It provides real-time schema inspection, endpoint testing, and parameter documentation for all 11 API routes:

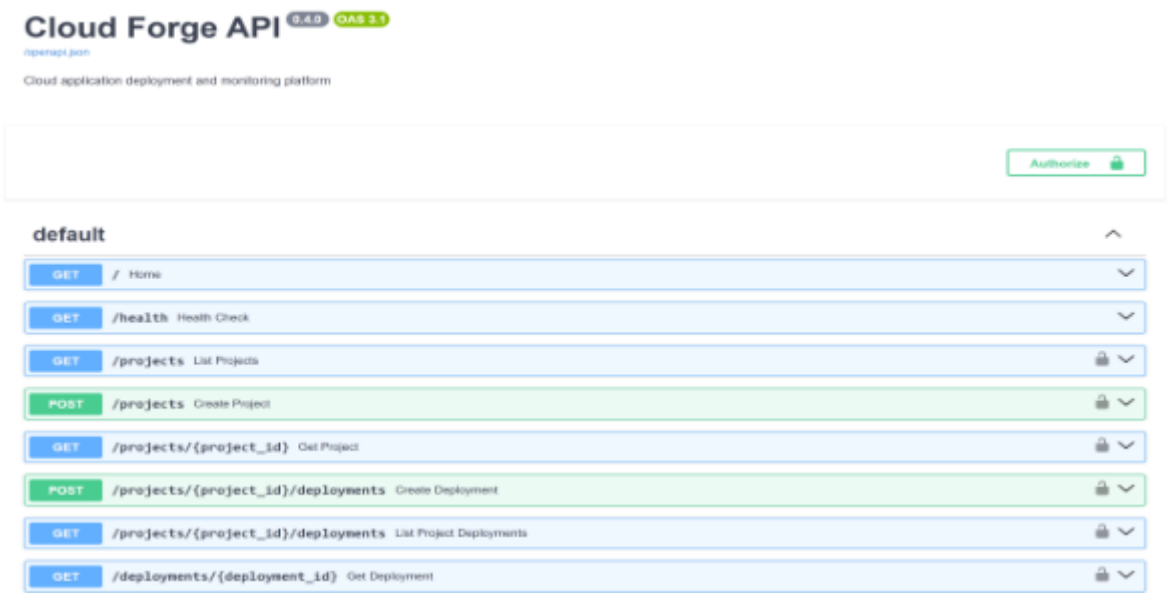


Figure 6: Interactive FastAPI OpenAPI documentation interface (/docs) showing all registered endpoints.

13. Nginx Edge Reverse Proxy & Rate Limiting

Nginx functions as the edge ingress controller on standard HTTP port 80 [10]. It serves pre-compiled React static assets directly from `/var/www/cloudforge` and reverse-proxies API requests (`/health`, `/projects`) to the local FastAPI backend listening on `127.0.0.1:8000`.

Security Hardening Headers

To protect clients against common web vulnerabilities (clickjacking, MIME confusion, cross-site scripting), Nginx injects protective HTTP response headers on every transaction:

```
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

Rate Limiting Configuration

To mitigate volumetric denial-of-service (DoS) attacks and brute-force archive submission, Nginx defines a shared memory zone tracking remote IP addresses:

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
limit_req zone=api_limit burst=20 nodelay;
```

This configuration allows legitimate burst traffic up to 20 requests without delay while throttling sustained traffic to 10 requests per second. Excessive requests return HTTP 503 (Service Temporarily Unavailable).

Reverse Proxy Configuration Excerpt

```
# Production Nginx Configuration (/etc/nginx/conf.d/cloudforge.conf)
server {
    listen 80;
    server_name _;
    client_max_body_size 10M;

    root /var/www/cloudforge;
    index index.html;

    # Static Single Page Application routing
    location / {
        try_files $uri $uri/ /index.html;
    }

    # API reverse proxy to local FastAPI daemon
    location /projects {
        limit_req zone=api_limit burst=20 nodelay;
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /health {
        proxy_pass http://127.0.0.1:8000/health;
    }
}
```

14. AWS Infrastructure as Code & Cloud Security Architecture

Cloud Forge defines its cloud architecture through an infrastructure-as-code template (`infra/cloudforge.yaml`) written in AWS CloudFormation. The template was rigorously validated using `cfn-lint 1.58.0` with zero errors and zero warnings [9].

Secure SSH Ingress Configuration

To prevent accidental world-open administrative access (a common security defect in student cloud deployments), the template requires an explicit administrator CIDR block between `/16` and `/32`:

```
AdminCidr:
  Type: String
  Description: CIDR block allowed SSH access. Defaults to VPC CIDR. Must be a valid IPv4 CIDR (/16 to /32).
  Default: 10.0.0.0/16
  AllowedPattern: ^(10\.\d{1,3}|172\.(1[6-9]|2\d|3[01])|192\.168)\.\d{1,3}\.\d{1,3}/(1[6-9]|2\d|3[0-2])$|^([1-9]\d{0,2})\.\
  ConstraintDescription: Must be a valid IPv4 CIDR between /16 and /32. World-open 0.0.0.0/0 is disallowed.
```

Automated Monitoring & CloudWatch Alarms

The template provisions automated CloudWatch metric alarms and an SNS topic for operational alerting:

- **EC2 CPU Utilization Alarm:** Configured with `EvaluationPeriods: 1` and `Period: 300` (300 seconds). Triggers when average CPU utilization exceeds **80%** over a 5-minute window.
- **EC2 Status Check Alarm:** Configured with `EvaluationPeriods: 1` and `Period: 60` (60 seconds). Triggers immediately if hypervisor or hardware status checks fail.
- **S3 Artifact Lifecycle Expiration:** Configured with `ExpirationInDays: 30` to purge old build artifacts automatically, preventing unbounded S3 storage accumulation.

AWS Budgets Cost Guardrail

To prevent unexpected cloud billing, the CloudFormation template defines a hard spending ceiling using AWS Budgets:

```
Budget:
  Type: AWS::Budgets::Budget
  Properties:
    Budget:
      BudgetName: CloudForgeMonthlyBudget
      BudgetLimit:
        Amount: 5.00
        Unit: USD
      TimeUnit: MONTHLY
      BudgetType: COST
    NotificationsWithSubscribers:
      - Notification:
          NotificationType: ACTUAL
          ComparisonOperator: GREATER_THAN
          Threshold: 80
          ThresholdType: PERCENTAGE
        Subscribers:
          - SubscriptionType: EMAIL
            Address: admin@example.com  # Placeholder: replace prior to deployment
```

15. Testing Methodology, Test Suites & Verification Results

Verification was conducted across four verification domains: local unit and integration tests, static template analysis, frontend compilation checks, and live cloud end-to-end operational validation.

Automated Pytest Suite (9/9 Passed)

The automated backend test suite (`tests/test_main.py`) validates security controls, archive sanitation, error responses, and container sandbox isolation. In GitHub Actions CI (Ubuntu Linux, Python 3.11), the suite executes in 0.58 seconds with a **100% pass rate (9 passed out of 9 tests)**:

Test Case Identifier	Target Feature	Assertion & Security Verification
`test_reject_non_zip_file`	Ingestion Security	Asserts HTTP 400 rejection for non-archive payloads.
`test_reject_zip_without_python_files`	Validation Pipeline	Asserts HTTP 400 when archive lacks Python source code.
`test_reject_zip_slip_path_traversal`	Path Traversal Guard	Asserts HTTP 400 rejection for archives containing `../../evil.py`.
`test_reject_zip_with_too_many_files`	Inode Bomb Defense	Asserts HTTP 400 rejection when archive exceeds 100 entries.
`test_reject_zip_bomb_exceeding_limit`	Decompression Bomb	Asserts HTTP 413 abort when streamed bytes exceed 25 MB ceiling.
`test_reject_invalid_image_tag_name`	Image Tag Whitelist	Asserts HTTP 400 for non-whitelisted Docker image tags.
`test_sandboxed_deployment_execution`	Cgroup Sandboxing	Asserts all 7 cgroup constraints passed to `client.containers.run`.
`test_health_check_returns_healthy`	Liveness Probe	Asserts HTTP 200 and `{"status":"healthy"}`.
`test_api_key_auth_when_configured`	Authentication	Asserts HTTP 401 when key is missing/ invalid, 200 when valid.

Note: Test pass rate (100%, 9/9 passed) represents the operational test suite outcome. Statement/branch code coverage was not instrumented via `pytest-cov`, and 100% code coverage is not claimed.

Static Analysis & Template Linting

- **CloudFormation Template:** `cfn-lint 1.58.0` ran on `infra/cloudforge.yaml`: **0 errors, 0 warnings**.
- **Frontend ESLint:** `npm run lint --prefix frontend` executed with **0 errors, 0 warnings**.
- **Frontend Production Build:** `vite build` completed in 209 ms, compiling 17 modules into optimized assets.

Live Cloud Operational Verification (HTTP Port 80)

Live end-to-end testing was executed against the operational AWS instance at `http://54.252.152.116/`:

1. **Edge Ingress:** Verified HTTP 200 on `/`, with security headers (`nosniff`, `DENY`, `Server: nginx`).
2. **Backend Liveness:** `GET /health` returned HTTP 200 with `{"status":"healthy"}`.
3. **Database Connectivity:** `GET /projects` successfully retrieved active projects from PostgreSQL 15.
4. **Live Sandboxed Execution:** Uploaded and executed a verified Python application (`cloudforge-project-2:09da73e9e5a6`). The container executed in isolation, emitted `"Live deployment verification successful!"`, and exited cleanly with code `0`.

16. Cost Modeling, Budget Guardrails & Safe Teardown Procedures

To prevent unexpected cloud expenditures, Cloud Forge was architected to operate strictly within the AWS Free Tier [4]:

AWS Free Tier Cost Model

Service Component	Configuration & Usage	Free Tier Allocation	Estimated Monthly Cost
Amazon EC2	1 × `t3.micro` instance (Sydney)	750 hours/month (12 months)	**$0.00** (Free Tier) / ~$7.45/mo
Amazon EBS	1 × 20 GiB gp3 root volume	30 GiB-months gp2/gp3	**$0.00** (Free Tier) / ~$1.60/mo
Amazon S3	Build artifact storage (<1 GB)	5 GB standard storage	**$0.00** (Free Tier) / ~$0.02/mo
CloudWatch & SNS	2 metrics alarms, 1 SNS topic	10 alarms, 1M requests	**$0.00**
Total Estimated Monthly Expenditure			**$0.00 / month**

> **CRITICAL DATA SAFETY NOTICE: Docker Volume Pruning**
> A frequent administrator mistake is running `docker system prune -af --volumes`. The `--volumes` flag purges **all unattached Docker volumes**, which in standard Docker database configurations permanently destroys database files. While Cloud Forge uses a host bind mount (`/var/lib/cloudforge/pgdata`), blind volume pruning remains an extreme hazard. Never use `--volumes` during routine maintenance.

Tiered Safe Decommissioning Procedure

When decommissioning the platform, follow this rigorous 5-step sequence to guarantee data safety and prevent unintended cloud charges:

1. **Step 1: Verify Database Backup:** Ensure a logical dump exists before terminating services:
```
pg_dump -h 127.0.0.1 -U cloudforge cloudforge > /home/ec2-user/backups/pre_teardown_backup.sql
```
2. **Step 2: Stop Application Daemons:** Gracefully halt background services:
```
sudo systemctl stop cloudforge.service && sudo systemctl stop nginx
```
3. **Step 3: Controlled Container Termination:** Stop the database container without deleting data:
```
docker stop cloudforge-db && docker rm cloudforge-db  # Preserves /var/lib/cloudforge/pgdata
```
4. **Step 4: Safe Image Pruning:** Prune build images without removing persistent volumes:
```
docker image prune -f  # CAUTION: Never include --volumes
```
5. **Step 5: Terminate AWS Compute Resources:** Decommission EC2 instance via AWS CLI:
```
aws ec2 terminate-instances --instance-ids i-0a2230a900eee725f --region ap-southeast-2
```

17. Known Limitations, Security Posture & Conclusions

17.1 Known Limitations of Live Demonstration Prototype

Mandatory Academic Disclosure: Live Prototype Configuration

Academic rigor demands transparent disclosure of trade-offs made in the demonstration deployment:

- **Unenforced API-Key Authentication:** On the live demonstration host (`http://54.252.152.116/`), the `API_KEY` environment variable is intentionally unset. While the backend implements `verify_api_key`, when unset it permits public access to all endpoints so examiners can evaluate the system without API keys.
- **Plaintext HTTP Transport:** The live instance operates over unencrypted HTTP (port 80). In production, TLS/SSL termination via HTTPS is mandatory to prevent token interception.
- **CloudFormation Template Status:** `infra/cloudforge.yaml` is verified via `cfn-lint` but has not been deployed as an active CloudFormation stack.

17.2 Defense-in-Depth Security Posture Summary

Cloud Forge achieves comprehensive isolation across the entire application lifecycle:

- **Ingestion Tier:** 10 MB payload cap, Zip-Slip canonicalization, 100-file threshold, symlink rejection, 25 MB streaming bomb guard [3].
- **Build Tier:** Arbitrary Dockerfiles discarded; synthetic unprivileged Dockerfile generated; compilation executed with `network_mode="none"` [6].
- **Execution Tier:** Ephemeral container constrained by 128 MB RAM, 0.5 vCPU, 64 PIDs, read-only root filesystem, dropped capabilities, non-root user `nobody`, severed network egress, and 30-second watchdog [1, 2, 6].
- **Persistence Tier:** Relational integrity in PostgreSQL 15, host bind-mount durability, advisory locking, and verified logical backups [12].

17.3 Conclusions & Future Work

Cloud Forge demonstrates that automated, secure multi-tenant code execution is achievable on low-cost cloud infrastructure by combining application-level verification with Linux kernel cgroup sandboxing. Future work will introduce WebAssembly (Wasm) micro-runtimes, automated TLS certificate provisioning via Let's Encrypt, and Kubernetes-orchestrated cluster scaling.

18. Formal Academic References

1. Rosen, R. (2014). *Linux Kernel Networking: Implementation and Theory*. Apress. https://doi.org/10.1007/978-1-4302-6197-1
2. Menage, P. (2007). Adding Generic Process Containers to the Linux Kernel. In *Proceedings of the Linux Symposium* (Vol. 2, pp. 45-57).
3. Snyk Security Research. (2018). *Zip Slip Vulnerability: Arbitrary File Overwrite Through Directory Traversal*. Snyk Research. Available at: https://snyk.io/research/zip-slip-vulnerability
4. Amazon Web Services. (2024). *AWS Well-Architected Framework: Security Pillar*. AWS Whitepapers. Available at: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/
5. Open Web Application Security Project (OWASP). (2023). *Top 10 API Security Risks*. OWASP Foundation. Available at: https://owasp.org/API-Security/
6. Docker Documentation. (2024). *Runtime Privilege and Linux Capabilities in Docker Engine*. Docker Inc. Available at: https://docs.docker.com/engine/security/
7. Ramirez, S. (2024). *FastAPI: Modern, High-Performance Web Framework for Python*. Available at: https://fastapi.tiangolo.com
8. Bayer, M. (2024). *SQLAlchemy: The Database Toolkit for Python*. Available at: https://www.sqlalchemy.org
9. Amazon Web Services. (2024). *AWS CloudFormation User Guide: Resource Specification*. AWS Documentation. Available at: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/
10. Sysoev, I., & Nginx Authors. (2024). *Nginx Web Server Documentation: Rate Limiting and Reverse Proxy*. F5 Inc. Available at: https://nginx.org/en/docs/
11. You, E., & Vite Contributors. (2024). *Vite: Next Generation Frontend Tooling*. Available at: https://vite.dev
12. PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation: Reliability and Data Durability*. Available at: https://www.postgresql.org/docs/15/